Software Requirements Specification

NexaLife – Your Life, Our Promise

A Full-Stack Insurance Lifecycle Platform

Document Title	Software Requirements Specification (SRS)
Project Name	NexaLife – Life Insurance Management System
Date	March 2026
Backend	ASP.NET Core 8 • C# • Entity Framework Core
Frontend	Angular 17 • TypeScript • Tailwind CSS
Database	SQL Server (LocalDB / Azure SQL)
Authentication	JWT Bearer Tokens • BCrypt Hashing
Architecture	Clean Architecture • RESTful API • Standalone Angular Components
User Roles	Admin • Agent • Claims Officer • Customer
Standard	IEEE Std 830-1998 — Recommended Practice for SRS

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document defines the functional, non-functional, and system requirements for NexaLife — a full-stack, enterprise-grade Life Insurance Management System. The document is prepared in conformance with IEEE Std 830-1998 and is intended for consumption by software architects, developers, QA engineers, project managers, and business stakeholders.

The primary purpose of NexaLife is to digitize and automate manual life insurance workflows — encompassing policy acquisition, underwriting, premium billing, policy servicing, and claims settlement — thereby eliminating paper-based processes, reducing operational latency, and improving auditability across all insurance lifecycle touchpoints.

1.2 Scope

NexaLife is a multi-tenant, role-aware web application that manages the end-to-end lifecycle of life insurance operations. The system scope covers the following functional domains:

- Product Management: Configuration of insurance plans, premium rate tables, eligibility rules, and coverage parameters by system administrators.
- Underwriting & Risk Assessment: Agent-driven evaluation of applicant risk profiles, premium computation using a formula-based underwriting engine, and policy approval workflows.
- Financial Billing & Collections: Automated invoice generation with support for monthly, quarterly, and annual premium frequencies; payment tracking; grace period enforcement; and lapse management.
- Policy Servicing (Endorsements): A structured mechanism enabling customers to request modifications to active policies (address, nominee, sum assured changes) with a JSON-based audit trail.
- Claims Management: Document-backed claim submission by nominees, investigation and evidence review by Claims Officers, and automated settlement processing.
- Commission Management: Automated calculation and tracking of agent commissions derived from policy premiums.
- System Administration: User account management, role assignment, insurance plan configuration, and system-wide analytics dashboards.

1.3 Definitions, Acronyms, and Abbreviations

Term / Acronym	Definition
SRS	Software Requirements Specification
SPA	Single Page Application (Angular Frontend)
API	Application Programming Interface
JWT	JSON Web Token — used for stateless authentication
RBAC	Role-Based Access Control

CQRS	Command Query Responsibility Segregation
EF Core	Entity Framework Core — ORM for SQL Server data access
Sum Assured	The guaranteed amount payable to the nominee upon a valid claim
Premium	Periodic payment made by the customer to maintain policy coverage
Endorsement	A formal modification request to an active insurance policy
Grace Period	A defined period post due-date during which payment is accepted without lapsing
Lapse	Policy status when a premium invoice remains unpaid beyond the grace period
Underwriting	The process of evaluating risk and determining premium amount
Risk Category	Classification of customer risk as Low, Medium, or High
BaseRatePer1000	The base premium rate applied per INR 1,000 of Sum Assured
NFR	Non-Functional Requirement
FR	Functional Requirement
DTO	Data Transfer Object

1.4 Document Conventions

Requirements are uniquely identified using the format [MODULE-TYPE-NNN] (e.g., FR-POL-001 for Functional Requirement, Policy Module, Requirement 001). Priority levels are defined as: MUST (mandatory), SHOULD (recommended), and MAY (optional).

1.5 References

- IEEE Std 830-1998: IEEE Recommended Practice for Software Requirements Specifications
- NexaLife System Abstract and Project Overview Document
- NexaLife UML Diagrams: Activity, Class, ER, Sequence, Use Case, and Workflow Diagrams
- Clean Architecture by Robert C. Martin (Robert C. Martin Series)
- Microsoft ASP.NET Core 8.0 Documentation — https://learn.microsoft.com/en-us/aspnet/core
- Angular 17 Documentation — https://angular.io/docs
- Entity Framework Core 8.0 Documentation — https://learn.microsoft.com/en-us/ef/core

2. Overall System Description

2.1 System Context

NexaLife operates as a centralized digital platform replacing fragmented, paper-based insurance workflows. It interfaces with end users through a responsive Angular SPA and exposes all business capabilities through a RESTful ASP.NET Core Web API. The system persists all operational data in a SQL Server relational database accessed through Entity Framework Core.

The system is designed for deployment in a cloud-ready environment and supports concurrent access by all four user roles: Admin, Agent, Customer, and Claims Officer. All cross-role interactions are mediated through the API layer, enforcing security and business rules consistently.

2.2 System Architecture Overview

NexaLife is built on Clean Architecture principles, organized into four discrete layers with strict dependency rules (outer layers depend on inner layers, never the reverse):

Layer	Project	Responsibilities
Domain	NexaLife.Domain	Core business entities (Policy, Claim, Invoice, etc.), enumerations (PolicyStatus, UserRole, ClaimStatus), domain events, and business rule constants. No external dependencies.
Application	NexaLife.Application	Use cases implemented as CQRS Commands and Queries, service interfaces, DTOs, validation logic (FluentValidation), and application-level business rules.
Infrastructure	NexaLife.Infrastructure	Concrete implementations of repository interfaces, EF Core DbContext, SQL Server migrations, JWT token service, file storage, notification dispatch, and commission calculation.
API	NexaLife.API	ASP.NET Core Web API controllers, middleware pipeline (authentication, exception handling), Swagger/OpenAPI configuration, and dependency injection setup.

2.3 User Roles & Personas

2.3.1 Admin

The Admin is the system operator responsible for platform configuration and staff management. Admins have unrestricted access to system settings and analytics.

- Creates and manages insurance plans (coverage, premium rules, tenures, eligibility)
- Provisions staff accounts for Agents and Claims Officers
- Assigns system roles (Agent, ClaimsOfficer) to users

- Monitors platform-wide analytics: active policies, lapsed policies, total premiums, claim statistics
- Manages global system configuration and settings
- Has no direct involvement in individual policy or claim workflows

2.3.2 Agent (Insurance Underwriter)

The Agent performs underwriting operations — the critical business function of evaluating customer risk and approving or rejecting policy applications. Agents earn commissions on approved and active policies.

- Reviews customer policy applications with supporting documentation
- Evaluates applicant eligibility based on age, income, occupation, and medical history
- Assigns a risk category: Low, Medium, or High
- Applies risk multipliers to compute the final premium using the underwriting engine formula
- Approves or rejects policy applications with mandatory justification on rejection
- Processes endorsement (policy modification) requests from customers
- Tracks earned commissions per policy

2.3.3 Customer (Policyholder)

The Customer is the primary consumer of the platform. Customers manage their entire insurance lifecycle through the self-service portal.

- Registers on the platform with personal, financial, and identity details
- Browses available insurance plans and views coverage details
- Submits policy applications for a selected plan
- Uploads supporting documents (Address Proof, Income Proof, Nominee ID Proof)
- Manages nominee details with allocation percentages (must sum to 100%)
- Pays premium invoices as per billing frequency (Monthly / Quarterly / Annual)
- Views payment schedule and premium billing history
- Files insurance claims upon occurrence of insurable events
- Requests policy endorsements (address updates, nominee changes, sum assured modifications)

2.3.4 Claims Officer

The Claims Officer is responsible for the investigation, verification, and settlement of insurance claims. They are the final decision authority in the claims lifecycle.

- Reviews submitted claims and associated documentation
- Verifies nominee identity, policy validity, and payment history
- Transitions claim status through the investigation workflow (Submitted → UnderReview → Approved/Rejected)
- Approves valid claims and specifies the settled amount
- Generates transaction references for settlement payments
- Rejects fraudulent or invalid claims with a mandatory written reason

2.4 Operating Environment

- Backend: ASP.NET Core 8.0 Web API hosted on IIS or Docker (Linux containers)
- Frontend: Angular 17+ SPA served via Nginx or Azure Static Web Apps
- Database: Microsoft SQL Server 2019+ with Entity Framework Core 8.0
- Authentication: JWT Bearer tokens with configurable expiry
- API Documentation: Swagger/OpenAPI 3.0 (Swashbuckle)
- Target Browsers: Chrome 110+, Firefox 110+, Edge 110+, Safari 16+

2.5 Design & Implementation Constraints

- MUST adhere to Clean Architecture — no cross-layer dependency violations permitted
- All API endpoints MUST be secured with JWT and RBAC; unauthenticated access is prohibited except for /auth/register and /auth/login
- Premium calculation MUST use the canonical underwriting engine formula (see FR-UND-002)
- Policy status transitions MUST follow the defined state machine — no arbitrary status jumps
- All financial records (invoices, payments, commissions) are immutable once created; corrections are recorded as new entries
- The frontend MUST be a Single Page Application using Angular with component-based architecture

3. Functional Requirements

3.1 Module 1 — Authentication & User Management

3.1.1 Customer Self-Registration [FR-AUTH-001]

The system SHALL provide a public registration endpoint allowing new customers to create accounts. The registration form SHALL collect and validate the following fields:

- Full Name (required, max 200 characters)
- Date of Birth (required; system validates minimum age of 18 years)
- Email Address (required; globally unique; RFC 5322 validated)
- Password (required; minimum 8 characters; stored as bcrypt hash)
- Bank Account Number and Bank Name (required for claim settlement)

3.1.2 Staff Provisioning by Admin [FR-AUTH-002]

The Admin SHALL be able to create accounts for Agents and Claims Officers from the administration panel. Staff accounts are not self-registerable. The Admin SHALL assign the role (Agent or ClaimsOfficer) at the time of account creation.

3.1.3 JWT-Based Authentication [FR-AUTH-003]

All users SHALL authenticate via a login endpoint (/api/auth/login). Upon successful credential validation, the system SHALL issue a signed JWT Bearer token containing the UserId, Email, and Role claims. The token SHALL be included in the Authorization header for all subsequent API requests.

3.1.4 Role-Based Access Control [FR-AUTH-004]

Every protected API endpoint SHALL enforce RBAC using the role claim embedded in the JWT. Unauthorized role access SHALL return HTTP 403 Forbidden. Role-to-dashboard routing is defined as follows:

Role	Dashboard Route
Admin	/dashboard/admin
Agent	/dashboard/agent
Customer	/dashboard/customer
Claims Officer	/dashboard/officer

3.2 Module 2 — Insurance Plan Management

3.2.1 Create Insurance Plan [FR-PLN-001]

The Admin SHALL be able to create new insurance plans with the following attributes:

- Plan Name and Description
- Plan Type (e.g., Term, Whole Life, Endowment)
- Base Rate Per 1000 (decimal) — used in premium calculation
- Minimum and Maximum Sum Assured (INR)
- Tenure Options (comma-separated list of years, e.g., '10,15,20,25,30')
- Commission Percentage (decimal) — used for agent commission calculation
- Coverage Value and Annual Benefits summary
- Minimum Age / Maximum Age / Maximum Sum Assured eligibility constraints
- Risk Multipliers: Low Risk Multiplier, Medium Risk Multiplier, High Risk Multiplier

3.2.2 Modify and Deactivate Plans [FR-PLN-002]

The Admin SHALL be able to update any attribute of an existing insurance plan. Plan deactivation SHALL not affect existing active policies associated with that plan. Deleted or deactivated plans SHALL be removed from the customer-facing plan catalogue.

3.2.3 Public Plan Catalogue [FR-PLN-003]

The system SHALL expose a public (authenticated, Customer role) endpoint allowing customers to browse available plans, view plan details, compare coverage parameters, and select a plan to apply for.

3.3 Module 3 — Policy Lifecycle Management

The policy lifecycle represents the central workflow of NexaLife. The following state machine governs all policy status transitions:

From Status	To Status	Trigger / Condition
(New)	**Draft**	Customer submits a policy application for an eligible plan
Draft	**Submitted**	All nominee and document details have been provided by the customer
Submitted	**Approved**	Agent performs underwriting and approves the application
Submitted	**Rejected**	Agent rejects the application with a documented reason
Approved	**Active**	Customer makes the first premium payment and invoice is settled
Active	**Lapsed**	Invoice remains unpaid beyond the configured grace period
Lapsed	**Active**	Customer makes a late payment that settles the overdue invoice
Active	**Settled**	A claim against this policy is approved and settled
Active	**Cancelled**	Customer or Admin initiates a policy cancellation

3.3.1 Policy Application Submission [FR-POL-001]

A Customer SHALL be able to submit a policy application by selecting a plan and providing: applicant age, annual income, occupation type, and Aadhaar reference. The system SHALL perform basic eligibility

validation (age within plan limits, income thresholds) before accepting the application. Policy status is set to Draft upon creation.

3.3.2 Document Upload [FR-POL-002]

The Customer SHALL upload the following supporting documents as part of policy profile completion: Address Proof, Income Proof, and Nominee ID Proof. Each document SHALL include a DocumentType and FilePath. Documents are stored with a DocumentStatus (Pending, Approved, Rejected). The Agent may flag documents for re-upload if invalid.

3.3.3 Nominee Management [FR-POL-003]

The Customer SHALL add one or more nominees to the policy. Each nominee SHALL include: Full Name, Relationship, Contact Number, and Allocation Percentage. The system SHALL enforce a hard validation rule: the sum of all nominee allocation percentages MUST equal exactly 100%. Partial nominee records SHALL prevent policy submission.

3.3.4 Agent Underwriting & Approval [FR-POL-004]

The Agent SHALL review applications with status 'Draft' or 'Submitted'. The Agent SHALL perform a risk assessment and assign a Risk Category (Low, Medium, or High). The system SHALL automatically calculate the premium using the underwriting engine (see Section 3.4). The Agent SHALL approve or reject the application. On approval, the system generates the first premium invoice and notifies the customer.

3.3.5 Policy Activation [FR-POL-005]

Policy status SHALL transition from Approved to Active upon successful processing of the first premium payment. Upon activation, the system SHALL: set ActiveFrom and ActiveTo dates based on tenure, generate the full premium billing schedule, generate a Policy Document, and calculate the Agent's commission.

3.3.6 Policy Lapse & Reinstatement [FR-POL-006]

The system SHALL automatically monitor invoice due dates. If an invoice remains in Unpaid status after the grace period expires, the policy status SHALL be changed to Lapsed and the customer SHALL be notified. A Lapsed policy MAY be reinstated if the outstanding invoice is paid within the reinstatement window.

3.4 Module 4 — Dynamic Underwriting Engine

3.4.1 Premium Calculation Formula [FR-UND-001]

The premium calculation is the core of the underwriting process. The Annual Premium SHALL be computed using the following canonical formula:

```
Annual Premium = (SumAssured / 1000) × BaseRatePer1000 × TenureYears ×
RiskMultiplier
```

Where RiskMultiplier is determined from the InsurancePlan configuration:

- Low Risk → Plan.LowRiskMultiplier (e.g., 1.0)

- Medium Risk → Plan.MediumRiskMultiplier (e.g., 1.25)
- High Risk → Plan.HighRiskMultiplier (e.g., 1.5)

3.4.2 Billing Frequency Adjustment [FR-UND-002]

Once the Annual Premium is calculated, the system SHALL compute the per-period invoice amount based on the customer's selected payment frequency:

- Annual: Invoice Amount = Annual Premium
- Quarterly: Invoice Amount = Annual Premium / 4
- Monthly: Invoice Amount = Annual Premium / 12

3.4.3 Risk Assessment Input Factors [FR-UND-003]

The Agent SHALL consider the following factors when assigning a risk category: applicant age, declared annual income, occupation hazard level, declared medical history or pre-existing conditions, and the sum assured requested relative to income. The system SHALL store the assigned risk category in the Policy entity and use it in all subsequent premium recalculations.

3.5 Module 5 — Financial Billing & Payments

3.5.1 Automated Invoice Generation [FR-FIN-001]

Upon policy activation, the system SHALL generate the complete invoice schedule for the entire policy tenure based on the billing frequency. Each invoice SHALL contain: InvoiceNumber, PolicyId, AmountDue, DueDate, PaymentFrequency, Period (year/quarter/month number), and InvoiceStatus (Unpaid, Paid, Overdue).

3.5.2 Payment Processing [FR-FIN-002]

The Customer SHALL be able to initiate payment against an open invoice. The system SHALL process the payment transaction, record AmountPaid, PaymentDate, TransactionId (system-generated reference), and PaymentStatus (Success, Failed). On successful payment, the associated invoice SHALL be marked as Paid. If the payment is the first for an Approved policy, the policy SHALL transition to Active.

3.5.3 Grace Period & Lapse Management [FR-FIN-003]

The system SHALL support configurable grace periods per payment frequency. If payment is not received by DueDate + GracePeriodDays, the invoice status changes to Overdue and the policy status transitions to Lapsed. The system SHALL dispatch a notification to the customer upon invoice due date, at the midpoint of the grace period, and upon lapse.

3.5.4 Agent Commission Calculation [FR-FIN-004]

Upon policy activation, the system SHALL automatically calculate and record the Agent's commission. The formula is:

```
Commission Amount = Annual Premium × (Plan.CommissionPercentage / 100)
```

A Commission record SHALL be created with: PolicyId, AgentId, PremiumAmount, CommissionAmount, and EarnedAt timestamp. Agents SHALL be able to view their commission history via the Agent dashboard.

3.5.5 Payment Schedule View [FR-FIN-005]

The Customer SHALL be able to view the complete payment schedule for any active policy, showing all invoices with their due dates, amounts, and current status. Overdue invoices SHALL be visually highlighted.

3.6 Module 6 — Claims Management

The claims module governs the lifecycle of insurance claims from initial filing through final settlement or rejection.

From Status	To Status	Trigger
(New)	**Submitted**	Nominee files a claim against an active policy
Submitted	**UnderReview**	Claims Officer begins investigation
UnderReview	**Approved**	Officer verifies eligibility and approves settlement
UnderReview	**Rejected**	Officer determines claim is invalid; provides written reason
Approved	**Settled**	Settlement amount is disbursed to nominee bank account

3.6.1 Claim Filing [FR-CLM-001]

A registered nominee (Customer acting on behalf of the policy) SHALL be able to file a claim. The claim submission SHALL require: Cause (reason for claim), Date of Event (must be after Policy ActiveFrom date), Claim Amount (must not exceed Sum Assured), and nominee bank account details for settlement. The system SHALL validate that the policy is Active and the claiming party matches a registered nominee.

3.6.2 Claim Document Upload [FR-CLM-002]

Following claim submission, the nominee SHALL upload evidentiary documents stored in ClaimDocuments (separate from policy documents): Death Certificate (mandatory for mortality claims), Medical Reports, Hospital Records, and any Additional Supporting Evidence.

3.6.3 Claims Officer Review [FR-CLM-003]

The Claims Officer SHALL access a queue of submitted claims. Upon opening a claim for investigation, the status SHALL transition to UnderReview. The officer SHALL verify: nominee identity against policy records, policy validity and active status, payment compliance history, and sufficiency of uploaded claim documents.

3.6.4 Claim Decision & Settlement [FR-CLM-004]

Upon completing investigation, the Claims Officer SHALL: Approve — specify the settled amount (≤ claim amount), generate a transaction reference (format: TXNCLM-{ClaimId}-{YYYYMMDD}), update claim status to Settled, and record settlement metadata. On Rejection — provide a mandatory written rejection reason and update claim status to Rejected. The customer SHALL be notified of the claim decision via the notification system.

3.6.5 Policy Closure After Claim Settlement [FR-CLM-005]

Upon successful claim settlement, the system SHALL automatically transition the associated policy status to Settled (Closed). No further claims SHALL be accepted against a settled policy. If all policies of a customer are settled/closed, the customer's account status SHALL transition to Inactive, preventing new policy applications.

3.7 Module 7 — Policy Endorsements (Servicing)

3.7.1 Endorsement Request Submission [FR-END-001]

A Customer SHALL be able to submit policy modification requests for any Active policy. Supported endorsement types are:

- Address Change — Update the policyholder's registered address
- Nominee Update — Add, remove, or modify nominee details (allocation percentages must sum to 100% post-change)
- Sum Assured Change — Request an increase or decrease in coverage amount (subject to plan limits)

Each endorsement request SHALL capture: EndorsementType, ChangeRequestJson (the proposed new values in JSON format), OldValueJson (the current values at time of request — for audit trail), and EndorsementStatus (Pending, Approved, Rejected).

3.7.2 JSON-Based Audit Trail [FR-END-002]

The system SHALL persist both the previous and proposed values of each endorsement request as JSON blobs. This provides a complete, immutable audit trail of all policy modifications. The OldValueJson SHALL be captured by the system at the time of request submission, not provided by the customer.

3.7.3 Agent Endorsement Review [FR-END-003]

Agents SHALL access a queue of pending endorsement requests. The Agent SHALL review the proposed change against policy terms and plan constraints. On Approval, the system SHALL apply the change to the live policy record and update the endorsement status to Approved. On Rejection, the agent SHALL provide a reason and the endorsement status is set to Rejected. The customer SHALL be notified of the outcome.

3.8 Module 8 — Notifications

3.8.1 System Notifications [FR-NOT-001]

The system SHALL maintain a Notifications table that records all system-generated alerts for users. Each notification SHALL include: UserId, Message, IsRead flag, and CreatedAt timestamp. Notifications SHALL be triggered for the following events:

- Policy application approved or rejected
- Invoice generated and due soon
- Invoice overdue / policy lapsed
- Claim status updated (submitted, under review, settled, rejected)
- Endorsement request approved or rejected
- Document re-upload requested by agent

4. Technical & Architectural Requirements

4.1 Backend Architecture (ASP.NET Core 8.0)

4.1.1 API Layer [TR-API-001]

- RESTful API following HTTP semantics (GET, POST, PUT, PATCH, DELETE)
- All responses use consistent JSON envelope: { success, data, message, errors }
- Global exception handling middleware: catches unhandled exceptions, logs errors, and returns standardized HTTP error responses
- Swagger/OpenAPI 3.0 documentation with JWT authorization support
- Versioned API routes (e.g., /api/v1/policy)

4.1.2 Application Layer [TR-APP-001]

- Business logic encapsulated in Service classes or CQRS Command/Query Handlers
- AutoMapper for bidirectional DTO ↔ Entity mapping
- FluentValidation for input validation with rich error messages
- No direct dependency on Infrastructure — communicates through interfaces defined in Application layer

4.1.3 Infrastructure Layer [TR-INF-001]

- Entity Framework Core 8.0 as the ORM with SQL Server provider
- Repository pattern implementing IRepository<T> generic interface
- Database migrations managed by EF Core CLI (dotnet ef migrations)
- Automated data seeding: default Admin account, sample insurance plans, and system configuration
- JWT Token Service: generates and validates HS256/RS256 signed tokens
- File storage service for document uploads (local disk with configurable cloud storage extension point)

4.1.4 Domain Layer [TR-DOM-001]

- Pure C# entities with no external library dependencies
- Entities: User, Policy, InsurancePlan, Invoice, Payment, Nominee, Document, Claim, ClaimDocument, Commission, PolicyEndorsement, Notification
- Enumerations: UserRole, PolicyStatus, InvoiceStatus, ClaimStatus, EndorsementType, EndorsementStatus, PaymentFrequency, DocumentStatus, PaymentStatus
- Business rule constants defined as domain-layer static classes

4.2 Backend API Controllers

Controller	Responsibility
AuthController	Handles user registration, login, and JWT token issuance

DashboardController	Provides aggregated KPI metrics per role (admin, agent, customer, officer)
AdminController	Staff account creation, role management, and system administration operations
PolicyController	Full policy lifecycle: create, submit, underwrite, activate, view, cancel
ClaimController	Claim submission, document upload, officer review, approval, and settlement
EndorsementController	Endorsement request submission, agent review, and policy record updates
PaymentController	Invoice listing, payment processing, and payment history retrieval
NotificationsController	Fetch unread notifications, mark as read, and clear notifications

4.3 Frontend Architecture (Angular 17+)

- Single Page Application (SPA) with component-based architecture and lazy-loaded feature modules
- Angular Router with role-based route guards (CanActivate) enforcing RBAC at the UI level
- HTTP Interceptors for: automatic JWT header injection, global error handling, and loading state management
- Reactive Forms with server-side error integration for all data entry workflows
- Tailwind CSS utility classes for responsive, mobile-first UI design
- Real-time notification polling or SignalR integration for live system alerts
- Shared component library: notification bell, status badge, document uploader, invoice table

4.4 Security Requirements

4.4.1 Authentication [SEC-001]

- All API endpoints except /api/auth/register and /api/auth/login MUST require a valid JWT Bearer token
- JWT tokens SHALL contain: UserId (sub), Email, Role, and Expiry (exp) claims
- Token expiry SHALL be configurable (default: 24 hours); refresh token mechanism is RECOMMENDED

4.4.2 Authorization [SEC-002]

- RBAC enforced at controller/action level using [Authorize(Roles = '...')] attributes
- Customers SHALL NOT access agent, admin, or officer endpoints
- Agents SHALL NOT access admin or officer endpoints
- Data-level authorization: customers SHALL only access their own policies, claims, and invoices

4.4.3 Password Security [SEC-003]

- All passwords MUST be hashed using BCrypt (work factor ≥ 12) before storage
- Plain-text passwords SHALL NEVER be stored or logged
- Password reset flow SHALL use time-limited, single-use tokens sent to the registered email

4.4.4 Input Security [SEC-004]

- All API inputs SHALL be validated using FluentValidation before processing
- SQL injection is mitigated by exclusive use of parameterized EF Core LINQ queries
- All file uploads SHALL be validated for type (whitelist: PDF, JPEG, PNG) and maximum size (configurable, default 10MB)

5. Non-Functional Requirements

ID	Category	Requirement
NFR-001	Performance	API response time SHALL be ≤ 500ms for 95% of requests under normal load (≤ 100 concurrent users). Dashboard queries SHALL return within 1 second.
NFR-002	Scalability	The system SHALL be designed for horizontal scaling. The API tier SHALL be stateless to enable load balancing. Database connection pooling SHALL be configured.
NFR-003	Reliability	The system SHALL achieve 99.5% uptime during business hours. All financial transactions (payments, settlements) SHALL be wrapped in database transactions to ensure ACID compliance.
NFR-004	Maintainability	Strict Clean Architecture adherence is mandatory. No business logic SHALL reside in controllers. All services SHALL be dependency-injected. Code coverage for unit tests SHALL be ≥ 80% for Application layer services.
NFR-005	Auditability	All policy endorsement changes SHALL record OldValueJson and ChangeRequestJson. Payment and claim settlement records are immutable. Status transitions SHALL be logged with timestamps and the acting user's ID.
NFR-006	Data Integrity	Policy status transitions SHALL enforce the defined state machine — invalid transitions SHALL be rejected with HTTP 422. Nominee allocation percentages SHALL always be validated to sum to exactly 100%.
NFR-007	Security	Passwords stored as BCrypt hashes (factor ≥ 12). JWT tokens signed with HS256 or RS256. All endpoints enforcing RBAC. File uploads restricted to PDF, JPEG, PNG formats ≤ 10MB.
NFR-008	Usability	The Angular SPA SHALL be fully responsive across desktop (1920×1080), tablet (768×1024), and mobile (375×667) breakpoints. Key workflows SHALL complete in ≤ 5 user interactions.
NFR-009	Testability	All Application layer services SHALL have corresponding xUnit unit tests with Moq-based dependency mocking. Integration tests SHALL cover critical API endpoints.
NFR-010	Documentation	All API endpoints SHALL be documented in Swagger/OpenAPI with request/response schemas, authentication requirements, and example values. Code SHALL include XML documentation comments.

6. Data Requirements

6.1 Core Entity Descriptions

The following describes each core domain entity, its purpose, and its key attributes as derived from the system's Class Diagram and ER Diagram.

6.1.1 User

Represents all platform participants across all roles. A single Users table supports polymorphic roles via the UserRole enumeration.

Attribute	Type	Description
Id (PK)	int	System-generated primary key
FullName	string	Legal full name of the user
Email	string	Unique login identifier (RFC 5322)
PasswordHash	string	BCrypt-hashed credential
UserRole	enum	Admin | Agent | Customer | ClaimsOfficer
IsActive	bool	Account enabled/disabled flag
DateOfBirth	DateTime	Used for age-based eligibility validation
BankAccountNumber	string	Nominee bank account for claim settlements

6.1.2 InsurancePlan

Master product catalogue entity. Defines the terms, rates, and constraints for each insurance product offered on the platform.

Attribute	Type	Description
Id (PK)	int	System-generated primary key
PlanName	string	Commercial name of the plan
Description	string	Marketing description of coverage benefits
BaseRatePer1000	decimal	Core premium rate applied per INR 1,000 of sum assured
MinSumAssured / MaxSumAssured	decimal	Allowable coverage range
TenureOptions	string	Comma-separated list of allowed tenure years
CommissionPercentage	decimal	Agent commission rate as a percentage of annual premium

LowRiskMultiplier / MediumRiskMultiplier / HighRiskMultiplier	decimal	Premium adjustment factors per risk category
MinAge / MaxAge	int	Applicant eligibility age range

6.1.3 Policy

The central transactional entity representing an individual insurance contract. A Policy links a Customer (User), an InsurancePlan, and an Agent together.

Attribute	Type	Description
Id (PK)	int	System-generated primary key
PolicyNumber	string	Human-readable unique identifier (system-generated)
PolicyStatus	enum	Draft | Submitted | Approved | Active | Lapsed | Settled | Cancelled | Rejected
SumAssured	decimal	Coverage amount selected by customer
PremiumAmount	decimal	Final calculated annual premium (set during underwriting)
TenureYears	int	Policy duration in years
ActiveFrom / ActiveTo	DateTime	Policy coverage start and end dates (set on activation)
RiskCategory	string	Low | Medium | High (assigned by agent)
CustomerId / AgentId / InsurancePlanId	int (FK)	Foreign key references to related entities

6.1.4 Invoice

Represents a single billing period payment obligation. A policy generates multiple invoices based on its payment frequency and tenure.

Attribute	Type	Description
Id (PK)	int	System-generated primary key
InvoiceNumber	string	Unique human-readable invoice reference
PolicyId	int (FK)	Parent policy reference
AmountDue	decimal	Amount payable for this billing period
DueDate	DateTime	Payment deadline
PaymentFrequency	enum	Monthly | Quarterly | Annual
Period / PeriodNumber	int	The billing period number within the policy schedule
InvoiceStatus	enum	Unpaid | Paid | Overdue

6.1.5 Payment

Records a payment transaction against a specific invoice. Payments are immutable once recorded.

Attribute	Type	Description
Id (PK)	int	System-generated primary key
InvoiceId	int (FK)	The invoice this payment settles
PolicyId	int (FK)	Denormalized for quick policy payment history queries
AmountPaid	decimal	Actual amount received
PaymentDate	DateTime	Timestamp of successful transaction
TransactionId	string	System-generated unique transaction reference
PaymentStatus	enum	Success | Failed | Pending

6.1.6 Nominee

Represents a beneficiary associated with a specific policy. Multiple nominees may be registered per policy with fractional claim allocation.

Attribute	Type	Description
Id (PK)	int	System-generated primary key
PolicyId	int (FK)	Parent policy reference
FullName	string	Legal name of the nominee
Relationship	string	Relationship to policyholder (e.g., Spouse, Child, Parent)
ContactNumber	string	Nominee contact number for claim communication
AllocationPercentage	decimal	Percentage of sum assured payable to this nominee (all nominees must sum to 100%)

6.1.7 Document

Stores references to supporting documents uploaded by the customer as part of the policy application process.

Attribute	Type	Description
Id (PK)	int	System-generated primary key
PolicyId	int (FK)	Parent policy reference
DocumentType	string	Address Proof | Income Proof | Nominee ID Proof
FileName / FilePath	string	Original filename and server storage path
DocumentStatus	enum	Pending | Approved | Rejected

RejectionNotification	string	Agent's reason for rejection (if applicable)

6.1.8 Claim

Represents an insurance claim filed against an active policy. Claims are processed exclusively by Claims Officers.

Attribute	Type	Description
Id (PK)	int	System-generated primary key
ClaimNumber	string	Unique human-readable claim reference
PolicyId / CustomerId / ClaimsOfficerId	int (FK)	Relational references
Reason / OfficerRemarks	string	Cause of claim and officer investigation notes
ClaimAmount	decimal	Amount claimed by the nominee
SettledAmount	decimal	Actual amount disbursed (set on approval)
ClaimStatus	enum	Submitted | UnderReview | Approved | Settled | Rejected
BankName / BankAccountNumber	string	Nominee bank details for settlement disbursement
TransferReference	string	Settlement transaction reference (TXNCLM-{Id}-{YYYYMMDD})

6.1.9 Commission

Records the agent's earned commission for each activated policy. Created automatically upon policy activation.

Attribute	Type	Description
Id (PK)	int	System-generated primary key
PolicyId / AgentId	int (FK)	Relational references
PremiumAmount	decimal	Annual premium amount used as base for commission calculation
CommissionAmount	decimal	Calculated commission = PremiumAmount × CommissionPercentage
EarnedAt	DateTime	Timestamp of commission calculation (policy activation date)

6.1.10 PolicyEndorsement

Tracks customer-initiated modification requests for active policies with full before/after audit trail.

Attribute	Type	Description
Id (PK)	int	System-generated primary key
PolicyId / RequestedByCustomerId / ReviewedByAgentId	int (FK)	Relational references
EndorsementType	enum	AddressChange | NomineeUpdate | SumAssuredChange
ChangeRequestJson	string (JSON)	Proposed new values in JSON format
OldValueJson	string (JSON)	System-captured current values at time of request (immutable audit record)
EndorsementStatus	enum	Pending | Approved | Rejected
RejectionReason	string	Agent's reason for rejection (if applicable)

7. Use Case Summary

The following table provides a consolidated summary of all use cases organized by role and functional module, as derived from the NexaLife Use Case Diagram.

Actor	Module	Use Case	Includes / Extends
Agent	Underwriting & Sales	Review Policy Applications	—
Agent	Underwriting & Sales	Perform Risk Assessment	«include» Calculate Premium
Agent	Underwriting & Sales	Approve / Reject Application	—
Agent	Underwriting & Sales	Review Endorsement Requests	—
Agent	Underwriting & Sales	Track Commissions	—
Admin	System Administration	Manage User Accounts (Staff)	—
Admin	System Administration	Configure Insurance Plans	—
Admin	System Administration	Monitor System Analytics	—
Admin	System Administration	Manage Global Settings	—
Customer	Policy & Finance	Browse Insurance Plans	—
Customer	Policy & Finance	Apply for Policy	«include» Upload Documents, «include» Manage Nominees
Customer	Policy & Finance	View Invoices & Pay Premium	—
Customer	Policy & Finance	File Insurance Claim	—
Customer	Policy & Finance	Request Policy Endorsement	—
Claims Officer	Claims Management	Review Claim Documents	—
Claims Officer	Claims Management	Approve / Reject Claims	«include» Process Settlement

8. Appendices

8.1 Policy Status State Machine

The following defines all valid policy status transitions. Any transition not listed below is invalid and SHALL be rejected by the system with an HTTP 422 Unprocessable Entity response.

- (New Application) → Draft: Initial state upon policy creation
- Draft → Submitted: Customer completes all required nominee and document information
- Submitted → Approved: Agent underwrites and approves application
- Submitted → Rejected: Agent rejects application (irreversible — customer must reapply)
- Approved → Active: First invoice payment is successfully processed
- Active → Lapsed: Overdue invoice remains unpaid beyond grace period
- Lapsed → Active: Customer settles outstanding overdue invoice
- Active → Settled: Claim against policy is approved and settled by Claims Officer
- Active → Cancelled: Policy voluntarily cancelled by Customer or Admin

8.2 Claim Status State Machine

- (New Claim) → Submitted: Nominee files claim with required details
- Submitted → UnderReview: Claims Officer begins formal investigation
- UnderReview → Approved: Officer validates all criteria and approves claim
- UnderReview → Rejected: Officer determines claim is invalid or fraudulent
- Approved → Settled: Settlement amount disbursed and transaction reference generated

8.3 System Transaction Reference Formats

Reference Type	Format	Example
Policy Number	POL-{Year}-{5-digit sequence}	POL-2026-00142
Invoice Number	INV-{PolicyId}-{Period}	INV-1042-003
Payment Transaction	TXN-{InvoiceId}-{YYYYMMDD}	TXN-2201-20260312
Claim Number	CLM-{Year}-{4-digit sequence}	CLM-2026-0087
Claim Settlement	TXNCLM-{ClaimId}-{YYYYMMDD}	TXNCLM-1002-20260304

8.4 Requirement Traceability Matrix

The following matrix maps key functional requirements to their corresponding system modules and UML artifacts.

Req. ID	Description	Module	Use Case	Diagram
FR-AUTH-001	Customer Registration	Auth Module	Register Account	Sequence, Activity
FR-POL-001	Policy Application	Policy Lifecycle	Apply for Policy	Activity, Sequence
FR-POL-004	Agent Underwriting	Underwriting Engine	Perform Risk Assessment	Activity, Sequence
FR-UND-001	Premium Calculation	Underwriting Engine	Calculate Premium	Class, Activity
FR-FIN-001	Invoice Generation	Financial Module	View Invoices & Pay	ER, Class
FR-FIN-004	Commission Calculation	Financial Module	Track Commissions	Class, ER
FR-CLM-001	Claim Filing	Claims Management	File Insurance Claim	Sequence, Activity
FR-CLM-004	Claim Settlement	Claims Management	Approve/Reject Claims	Workflow
FR-END-001	Endorsement Request	Policy Servicing	Request Endorsement	Use Case
FR-NOT-001	System Notifications	Notifications	All Roles	Workflow

— End of Document —